Units 3 & 4 Implementation Report

Unsupervised Learning and Sequential Models,
built into the XAI Inventory & Procurement system

Team member	Register number	Focus
Sandhiya S	RA2411003010045	Clustering geometry & Markov chains
Nishanth Kannan	RA2411003010051	Mixtures, EM & hidden Markov models
Netti Nithin Chakravarthy	RA2411003010033	Latent dimensions & sequence decoding

What this document covers

Every topic listed in the Unit 3 and Unit 4 syllabi is implemented from first principles, run on the real 1,067,371-transaction UCI Online Retail II dataset, and verified. Each algorithm is pinned against a reference implementation (scikit-learn / SciPy) or against brute-force enumeration over all possible state paths, so that 'implemented from scratch' is a claim with evidence behind it. Every number in this report is read directly out of the JSON files the pipeline wrote; none of it is transcribed by hand.

Course: Machine Learning (21CSC305P) • Project: Explainable AI-Driven Decision Support for Predictive Inventory and Procurement Optimization in ERP Systems • Repository: github.com/sandhiya-s86/xai-inventory-procurement

1. What was added

The project already covered Units 1 and 2 (regression, classification, model selection, explainability). This work adds the complete Unit 3 and Unit 4 syllabi as two new subpackages, two new pipeline stages, two new dashboard pages and 158 new tests.

Deliverable	Unit 3	Unit 4
Source package	src/xaierp/unsupervised/ (8 modules)	src/xaierp/sequential/ (6 modules)
Pipeline stage	scripts/run_unsupervised.py	scripts/run_sequential.py
Result tables	10 CSV + findings JSON	12 CSV + findings JSON
Figures	6	4
Tests	79 passing	79 passing
Dashboard page	7 — SKU Segmentation	8 — Demand Regimes
Data artefact	data/processed/sku_segments.parquet	data/processed/demand_regimes.parquet

Headline result

The Unit 4 hidden Markov model finds **three demand regimes** the dataset never labels — QUIET, NORMAL and SURGE — and they align with the Christmas trading peaks in both years. Demand volatility differs **2.1x** between the calmest and busiest regime, which means a single pooled safety stock is simultaneously too small during SURGE and oversized during QUIET. That is a directly actionable finding neither the Unit 1 regression nor the Unit 2 classifier could have produced.

1.1 Syllabus coverage

Unit 3 topic	Where	Unit 4 topic	Where
K-means clustering	kmeans.py	Sequential data	sequential.py
Mixtures of Gaussians	gmm.py	Markov models	markov.py
EM algorithm	gmm.py	HMM	hmm.py
Alternative view of EM	em_lower_bound	Max. likelihood / EM	hmm.fit
Factor analysis	factor_analysis.py	Forward–backward	hmm.py
PCA	pca.py	Sum–product	sum_product.py
Choosing latent dims	choose_n_components	Scaling factors	forward_scaled
Measuring dissimilarity	dissimilarity.py	Viterbi	viterbi.py
Evaluating clustering	evaluation.py	Linear dynamical systems	lds.py
Hierarchical clustering	hierarchical.py	—	—

2. Unit 3 — Unsupervised learning on SKU demand profiles

The daily panel is 200 SKUs by 604 trading days. Clustering that directly would be clustering *SKU-days*, which answers no useful question. The panel is therefore collapsed to one row per SKU with 13 columns describing demand *behaviour* — volume, variability, intermittency, seasonality, weekly cycle, price and revenue share.

Leakage control

Profiles are computed on data up to **2011-05-15** — the end of the training split — and never on the test period. The segmentation is used downstream to set safety stock on the test window, so a profile built from the whole series would be a decision informed by its own answer. The test suite asserts that moving the cut-off actually changes the profiles, i.e. that the argument is not silently ignored.

2.1 Measuring dissimilarity

Clustering never sees the data, only the dissimilarity between pairs. The metric is the modelling assumption, so five were compared on the same standardised matrix. Spearman agreement is against Euclidean: it measures how much the *ranking* of 'which SKUs are alike' changes when the metric changes.

metric	mean distance	std distance	spearman vs euclidean
euclidean	4.382	2.633	1.000
manhattan	10.933	5.963	0.978
chebyshev	2.959	2.205	0.951
cosine	0.978	0.398	0.588
correlation	0.988	0.406	0.488

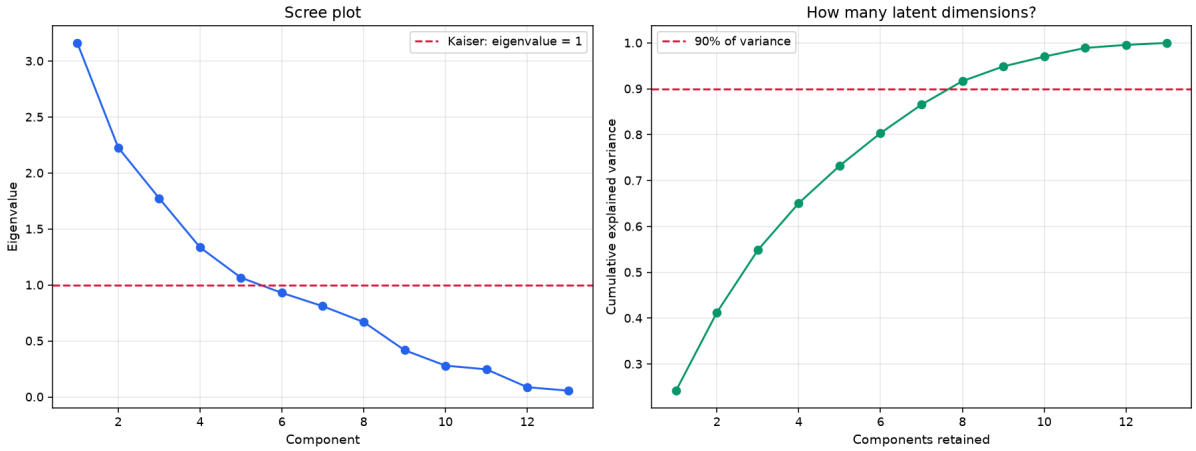
Euclidean and Manhattan rank pairs almost identically (rho = 0.978). Cosine and correlation do not (rho = 0.588 and 0.488) because they discard magnitude and compare only the *shape* of the profile — under them a slow seller and a fast seller with the same weekly pattern are neighbours. Euclidean on standardised features is used downstream, since replenishment volume genuinely matters for a stocking decision.

2.2 PCA and choosing the number of latent dimensions

PCA is computed by SVD of the centred matrix rather than by eigendecomposing the covariance, which squares the condition number. Components match scikit-learn to 1e-8 and are orthonormal to 1e-10.

Rule	Components	What it assumes
Kaiser (eigenvalue > 1)	5	A component must explain more than one standardised variable's worth
90% cumulative variance	8	A chosen threshold: tunable, but arbitrary
Scree elbow	1	Largest gap between consecutive eigenvalues

The three published rules disagree — 5, 8 and 1 components. All three are reported rather than quoting whichever is convenient. The pipeline uses Kaiser (L = 5), capturing **73.2%** of total variance at a reconstruction MSE of 0.2677. The elbow rule returning 1 is a known weakness of the geometric knee on a smoothly decaying spectrum, not a bug.



Scree plot and cumulative variance. The Kaiser line (eigenvalue = 1) and the 90% threshold give different answers, which is why both are reported.

2.3 Factor analysis, varimax and identifiability

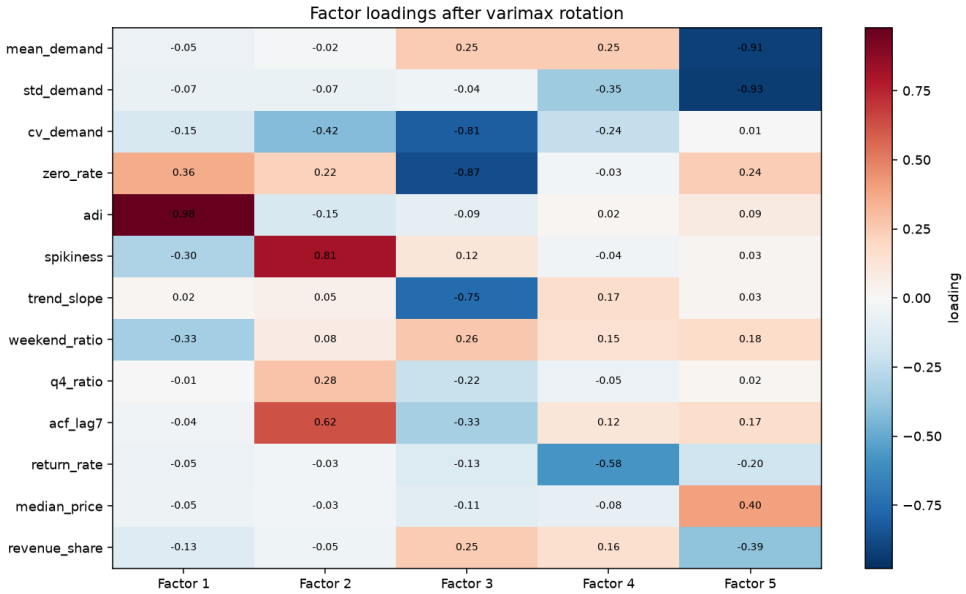
Factor analysis differs from PCA in a way worth stating precisely: PCA explains *total* variance and has no noise model, so it will happily spend a component on a column that is pure noise. FA explains only the *shared* variance, pushing the column-specific part into a diagonal noise matrix Psi.

```
x = W z + mu + eps,     z ~ N(0, I),  eps ~ N(0, Psi)
cov(x) = W W^T + Psi        (low rank + diagonal)
```

Quantity	Value
Factors fitted	5
EM iterations	2,378
Converged	True
Log-likelihood	-3,081.67
Log-likelihood non-decreasing every iteration	**True**
Free parameters (after removing rotations)	81
Mean communality	0.612

Rotational ambiguity, measured

For any orthogonal R, W' = WR gives an identical covariance and therefore an identical likelihood. Rotating the fitted loadings by a random orthogonal matrix moved individual loadings by up to **1.420** while the log-likelihood changed by **0.00e+00** and the implied covariance by 6.7e-16. W is identified only up to 10 free rotations, L(L-1)/2. This is exactly the licence under which varimax operates: rotating to a more interpretable solution costs nothing in fit, because every rotation fits equally well. Factor loadings must therefore never be reported as if they were unique.



Loadings after varimax rotation. Varimax maximises the variance of the squared loadings, pushing each towards 0 or +/-1 so that a factor loads strongly on few variables.

2.4 K-means and the elbow method

Lloyd's algorithm with k-means++ seeding and 20 restarts. Inertia matches scikit-learn to 1e-6 and the partition agrees at ARI = 1.000. Both of Lloyd's steps minimise inertia, so it can never increase — verified: **True** over 9 iterations, final inertia 1,447.96.

Inertia always falls as k rises (it reaches zero at k = n), so the minimum is never the answer. The elbow is found geometrically as the point furthest from the chord joining the ends of the curve — a reproducible rule rather than reading a chart by eye. It selects **k = 5**.

k	inertia	inertia drop pct
2	2,202.0	nan%
3	1,905.5	13.5%
4	1,723.9	9.5%
5	1,495.3	13.3%
6	1,349.9	9.7%
7	1,188.9	11.9%
8	1,001.6	15.8%
9	934.1	6.7%
10	846.6	9.4%

2.5 Mixtures of Gaussians, EM, and the alternative view

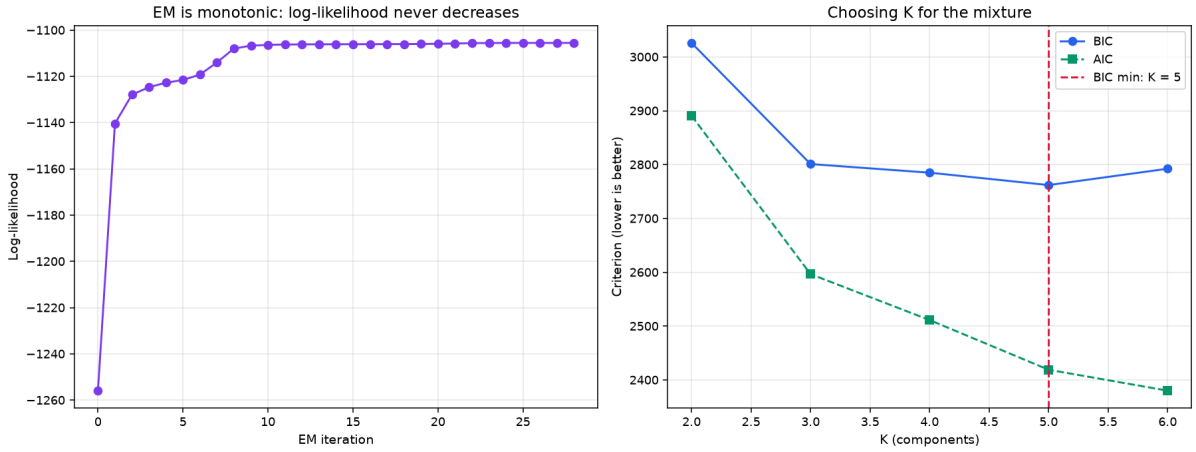
K-means assigns each SKU to exactly one segment with no measure of confidence. The Gaussian mixture is the soft version: each SKU gets a responsibility per component. The mixture is fitted in the PCA latent space because a full covariance over 13 dimensions needs about 91 parameters per component, which 200 SKUs cannot support.

Quantity	Value
Components selected by BIC	5
EM iterations	28
Log-likelihood, first iteration	-1,255.89
Log-likelihood, final	-1,105.51
Non-decreasing every iteration	**True**
Mean maximum responsibility	0.950
SKUs with no component above 90% responsibility	**31** of 200

The last row is the practical payoff of soft assignment. 31 SKUs sit genuinely between segments; K-means reports every one of them as confidently belonging to a single cluster, which for a stocking decision is a false certainty.

The alternative view of EM, verified numerically

EM can be read as coordinate ascent on the free energy F(q, theta) = E_q[log p(x,z|theta)] + H(q) = log p(x|theta) - KL(q || p(z|x,theta)). Because KL is non-negative, F is a lower bound on the log-likelihood, and the E-step makes it **tight**. Measured on the fitted mixture: at the true posterior the gap is **2.27e-13** (machine zero) with KL -3.6e-16; at a uniform q the gap is 1.821e+09 and equals the KL divergence exactly. That identity — gap = KL, zero only at the posterior — is the entire proof that EM cannot decrease the likelihood, and here it is a measurement rather than an assertion.



Left: the EM log-likelihood never decreases. Right: BIC has a genuine interior minimum, unlike inertia which only ever falls, so it can be used to choose K.

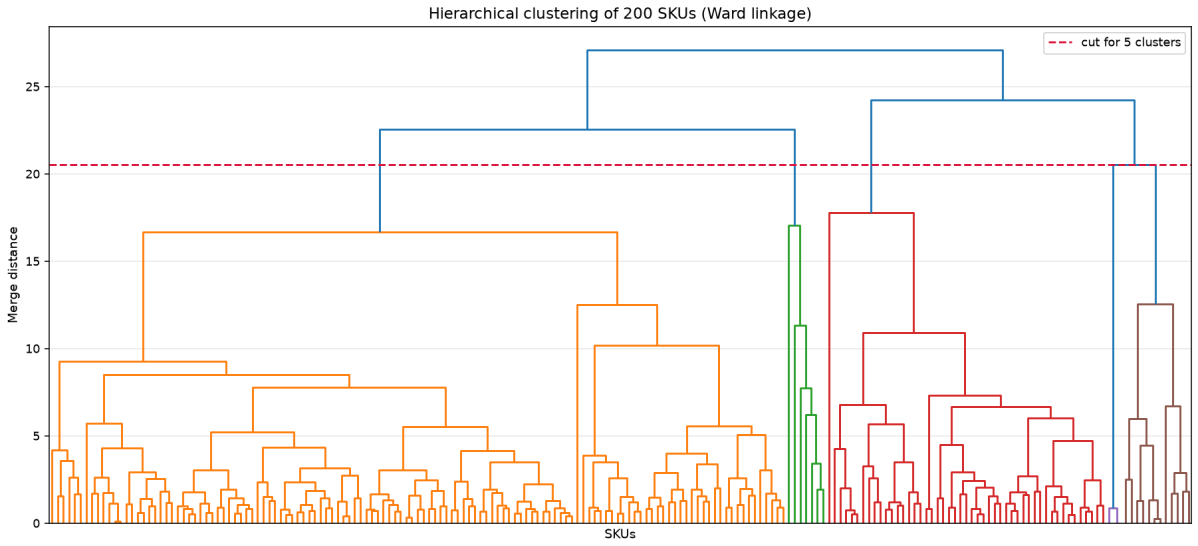
2.6 Hierarchical clustering

Four linkage methods implemented through one Lance–Williams update, producing a SciPy-format linkage matrix. All four match SciPy on merge heights and cluster sizes exactly, and the cophenetic correlations match SciPy to 1e-9.

linkage	cophenetic corr	silhouette	largest cluster share	smallest cluster size
single	0.879	0.594	97.0%	1
complete	0.850	0.527	93.5%	1
average	0.931	0.594	97.0%	1
ward	0.495	0.270	64.5%	3

A disagreement worth reporting rather than hiding

Average linkage scores the highest cophenetic correlation (0.931) — its dendrogram is the most faithful summary of the true pairwise distances — yet it places **97% of SKUs in a single cluster**. That is textbook chaining: a faithful tree is not the same thing as a useful partition. Ward scores the worst cophenetic correlation (0.495) and is nonetheless the only method producing balanced groups (largest cluster 64.5%), so the cut is taken from Ward. Reporting only the best cophenetic score would have been misleading.



Ward dendrogram over 200 SKUs; the dashed line is the cut giving 5 clusters.

2.7 Evaluating the output of clustering methods

Clustering is unsupervised, so there is no accuracy to report. Three internal indices and one relative index are used; all four match scikit-learn to 1e-9.

method	n clusters	silhouette	davies bouldin	calinski harabasz	largest cluster share
K-means	5	0.289	1.169	38.787	64.5%
GMM (hard assignment)	5	0.256	1.527	29.101	62.5%
Hierarchical (Ward)	5	0.270	1.188	37.192	64.5%

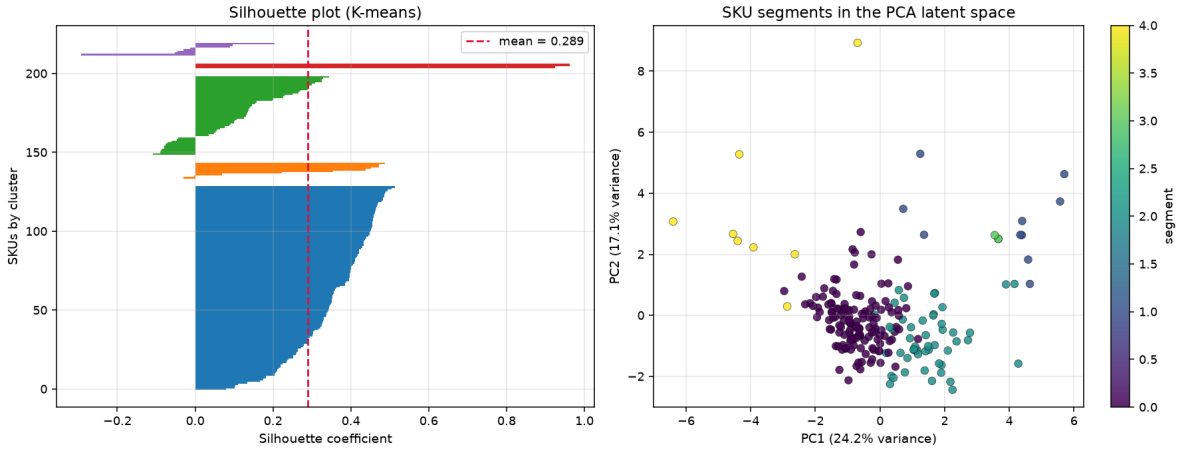
method a	method b	adjusted rand index
K-means	GMM (hard assignment)	0.596
K-means	Hierarchical (Ward)	0.839
GMM (hard assignment)	Hierarchical (Ward)	0.588

Silhouette and Calinski–Harabasz both reward compact spherical clusters, which is exactly what K-means produces by construction, so using them alone to declare K-means the winner would be close to circular. The Adjusted Rand Index is reported alongside: K-means and Ward agree strongly (ARI 0.839) while the mixture disagrees more (0.596), the expected consequence of a soft, covariance-aware model being allowed elliptical clusters.

2.8 The segments found

segment	segment name	n skus	mean demand	cv demand	adi	q4 ratio	revenue share total
0	High-volume sells-daily	129	28.8	2.26	1.31	1.59	64.2%
1	Mid-volume erratic	10	7.3	9.30	41.94	0.96	2.0%
2	Mid-volume weekly-cyclical	50	12.8	2.91	3.37	14.10	18.4%
3	Dormant (no sales in window)	3	0.0	0.00	427.00	0.00	0.0%
4	High-volume fast-moving	8	133.6	2.47	1.30	4.90	15.4%

Segments are named after the behavioural axis on which they are most extreme relative to the other segments, not by fixed thresholds. Segment 3 is a genuine finding rather than an error: three SKUs in the top 200 by lifetime revenue recorded no sales at all within the training window.



Silhouette plot and the segments in the PCA latent space.

3. Unit 4 — Sequential models on daily demand

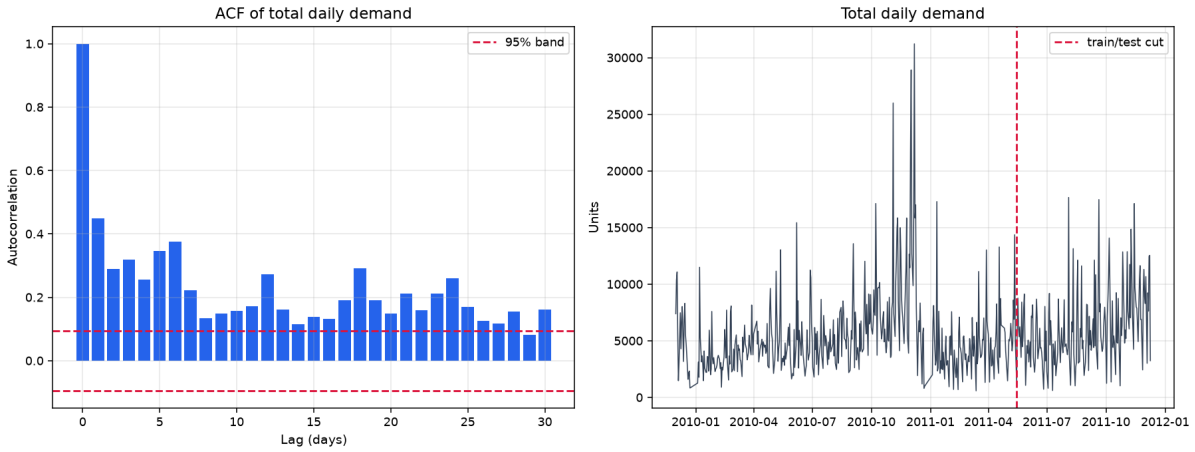
The business-wide daily series is 604 trading days, split chronologically into 427 training days and 177 held out after 2011-05-15.

3.1 Sequential data: does order carry information?

The whole unit rests on the claim that order matters. That is testable. The chain rule factorises the joint exactly; scoring the series under an i.i.d. model and under an AR(7) model measures how much the ordering is worth.

Quantity	Value	Reading
Significantly autocorrelated lags (of 30)	29	Almost every lag; strong dependence
ACF at lag 1	0.449	Yesterday predicts today
ACF at lag 7	0.222	A real weekly trading cycle
i.i.d. log-likelihood per day	-9.629	Order ignored
AR(7) log-likelihood per day	-9.450	Order used
Information gain	**0.179 nats/day**	The measured value of knowing the past
ADF statistic / p-value	-3.349 / 0.0128	Rejects a unit root
Block std ratio	2.52	Variance still drifts 2.5x across the series

The two stationarity readings disagree, and both are reported. ADF rejects a unit root, but it is blind to variance drift; splitting the series into blocks shows the standard deviation changing by a factor of 2.5. That heteroscedasticity is precisely what the regime-switching HMM in section 3.3 is built to represent.



ACF of total daily demand with the 95% white-noise band, and the raw series with the train/test cut.

3.2 Markov model of demand states

Daily demand is bucketed into four quantile states. The aggregate series never has a zero day, so all four states are quantile buckets; for a single SKU the discretiser gives zero its own state instead, because 31.7% of SKU-days are zeros and 'sold nothing' is a categorically different event from 'sold a little'. Cut-points are learned on the training window only.

from state	to LOW	to MEDIUM	to HIGH	to PEAK
LOW	0.406	0.245	0.236	0.113
MEDIUM	0.283	0.321	0.208	0.189
HIGH	0.215	0.271	0.308	0.206
PEAK	0.103	0.159	0.252	0.486

state name	empirical freq	stationary prob	self transition	expected run length
LOW	0.251	0.252	0.406	1.683
MEDIUM	0.248	0.249	0.321	1.472
HIGH	0.251	0.251	0.308	1.446
PEAK	0.251	0.247	0.486	1.945

Is the Markov structure real, or an artefact?

The buckets are equal-sized quantiles, so the stationary distribution comes out uniform *by construction* and proves nothing on its own — a point that is easy to miss. The real test is whether the diagonal beats what independence would predict. Persistence lift (self-transition divided by stationary probability) is 1.61x, 1.29x, 1.23x, 1.96x for LOW / MEDIUM / HIGH / PEAK, and a chi-square test of independence on the transition counts gives chi2 = 63.0 on 9 degrees of freedom, p = 3.53e-10. Busy days really do cluster: a PEAK day is almost twice as likely to be followed by another PEAK day as chance allows. That is why a buffer sized for an average day fails during a run of spikes.

Powers of A converge to the stationary distribution by k = 5 days, and the stationary vector satisfies pi = pi A to 5.6e-17. In other words the chain forgets its starting state within a working week — useful for a one-day-ahead question, useless for a one-month forecast, which is itself the motivation for the richer state-space models that follow.

3.3 HMM: learning demand regimes with Baum–Welch

A Markov chain over observed states assumes the thing that matters is the thing you can see. An HMM says the opposite: demand is driven by a hidden regime, and daily sales are a noisy emission from it. Three Gaussian-emission states were fitted on log1p demand by Baum–Welch with 8 random restarts.

Quantity	Value
Baum-Welch iterations	44
Converged	True
Log-likelihood, first iteration	-367.28
Log-likelihood, final	-329.20
Non-decreasing every iteration	**True**

label	mean demand units	self transition	expected duration days	stationary prob
QUIET	3,605	0.991	105.668	0.421
NORMAL	4,664	0.987	79.917	0.476
SURGE	9,244	0.942	17.277	0.103

The regimes are not an artefact of the fitting procedure: self-transitions of 0.94–0.99 mean they persist for weeks to months, and the Viterbi decoding places SURGE on the October to December trading peak in both 2010 and 2011. For a gift retailer that is the expected answer, arrived at without ever telling the model about Christmas.

3.4 Scaling factors

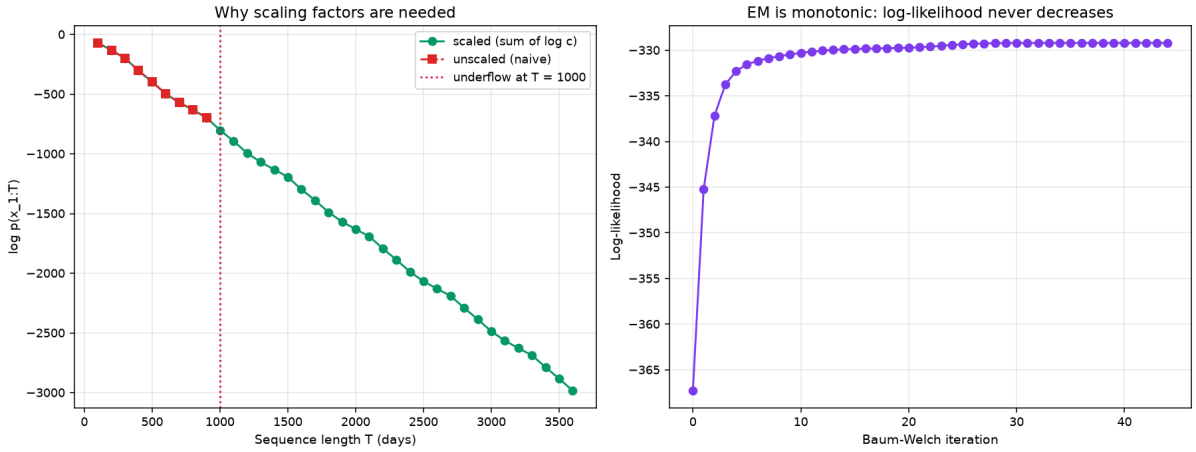
The forward recursion multiplies a probability at every step. For long sequences the running product underflows float64 to exactly zero, which silently turns the likelihood into -inf and every posterior into NaN. The fix is to normalise alpha at each step and keep the normaliser.

```
c_t = SUM_i alpha~_t(i)        alpha_t(i) = alpha~_t(i) / c_t

log p(x_1:T) = SUM_t log c_t
```

Measurement	Value
Unscaled log-likelihood at the real T = 604	-499.9988
Scaled log-likelihood at the same T	-499.9988
Did the real series underflow?	**No** — the raw product is about 1e-217, above the 1e-308 floor
Unscaled recursion still finite at	T = 900
Unscaled recursion collapses to -inf by	**T = 1000**
Scaled recursion still finite at	T = 3600
log p(x) equals the sum of log c_t	**True**
Scaled alpha sums to 1 at every t	**True**

Reporting only 'no underflow occurred' would have dodged the point, so the series was tiled to find the actual breaking point. The naive recursion survives 900 steps and is dead by 1,000; the scaled one is still finite at 3,600 because a sum of logarithms cannot underflow. At the real sequence length both agree to the last decimal place, which is the correct result — scaling changes the numerics, never the answer.



Left: scaled versus unscaled log-likelihood as T grows; the naive curve terminates where it underflows. Right: Baum-Welch log-likelihood, monotonically increasing.

3.5 Forward–backward: filtering versus smoothing

The forward pass alone gives the filtered posterior p(z_t | x_1:t), using the past only. Combining it with the backward pass gives the smoothed posterior p(z_t | x_1:T), which uses the whole series. They are different quantities and the difference is measurable.

Quantity	Value
gamma rows sum to 1	**True**
xi slices sum to 1	**True**
Days where filtering and smoothing disagree	**134** of 604 (22.2%)
Mean confidence, filtered	0.859
Mean confidence, smoothed	0.898

Smoothing is more confident because it conditions on more data. That is also why it is **not** admissible for a procurement decision: it uses observations from after the decision date. The safety-stock work in section 3.8 uses the filtered posterior only.

3.6 Viterbi versus posterior decoding

Viterbi is the forward algorithm with the sum replaced by a maximum, plus backpointers. It returns the single most probable path. Posterior decoding instead takes the most likely state at each time independently and never consults the transition matrix.

n timesteps	n disagreements	disagreement rate	viterbi logprob	posterior path logprob	logprob advantage viterbi	viterbi n switches	posterior n switches
604	21	3.5%	-514.622	-536.226	21.604	8	12

Why this distinction has teeth

On this series the two decoders disagree on 21 of 604 days (3.5%), and the Viterbi path is 21.6 log-units more probable. Posterior decoding also produces a twitchier labelling (12 regime switches against 8). The sharper failure mode is demonstrated in the test suite: because posterior decoding never looks at A, it can emit a sequence using a transition the model assigns probability zero — a path that is not merely unlikely but *impossible*. Viterbi is always a legal sequence.

3.7 Sum–product message passing

The notes state that forward–backward is sum–product on an HMM chain and that Viterbi is its max–product analogue. Rather than assert it, the general message-passing rules were implemented on a factor graph and the results compared.

max abs diff forward message vs alpha	max abs diff belief vs gamma	max abs diff pairwise vs xi	abs diff logpartition vs loglik	max product path matches viterbi
0.0e+00	0.0e+00	0.0e+00	0.0e+00	True

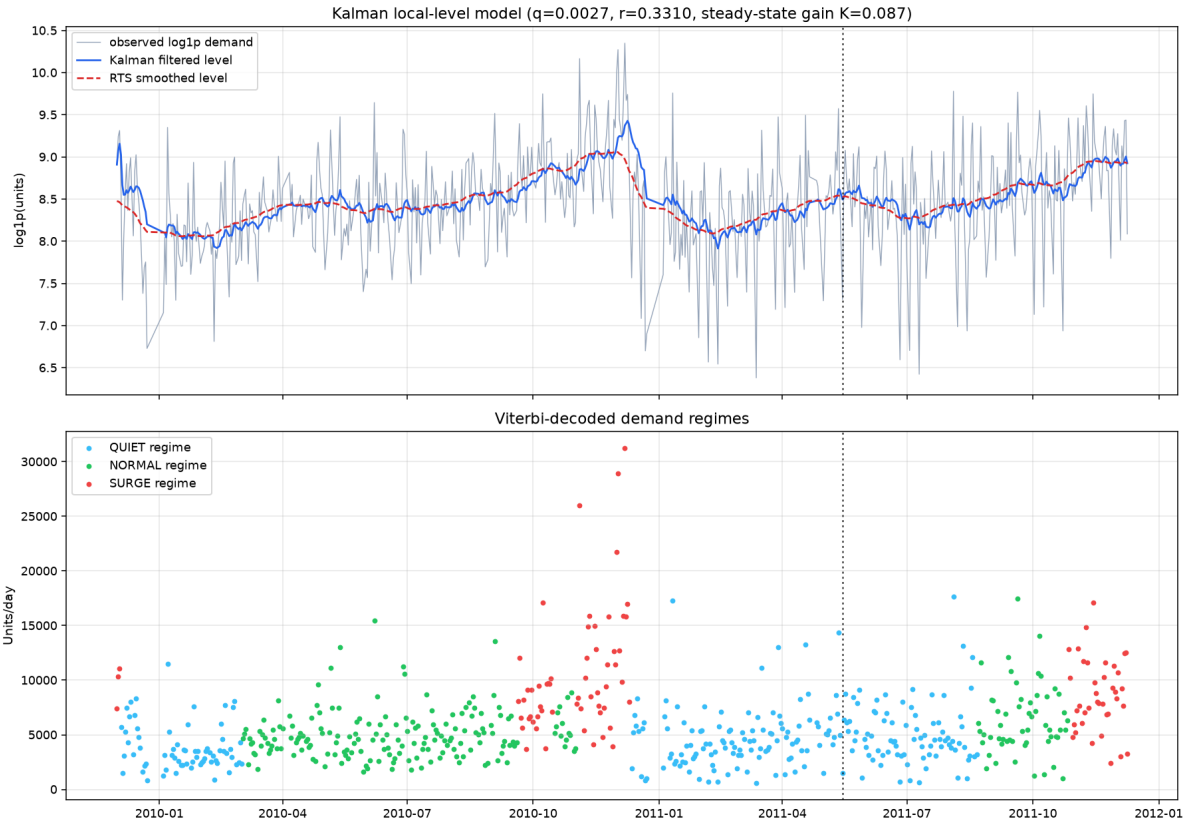
Every difference is exactly zero: the messages *are* alpha and beta, the beliefs *are* gamma, the pairwise beliefs *are* xi, and the log partition function *is* the forward log-likelihood. Swapping the summation for a maximum in the same code reproduces the Viterbi path exactly — the semiring view made concrete.

3.8 Linear dynamical system: Kalman filter and RTS smoother

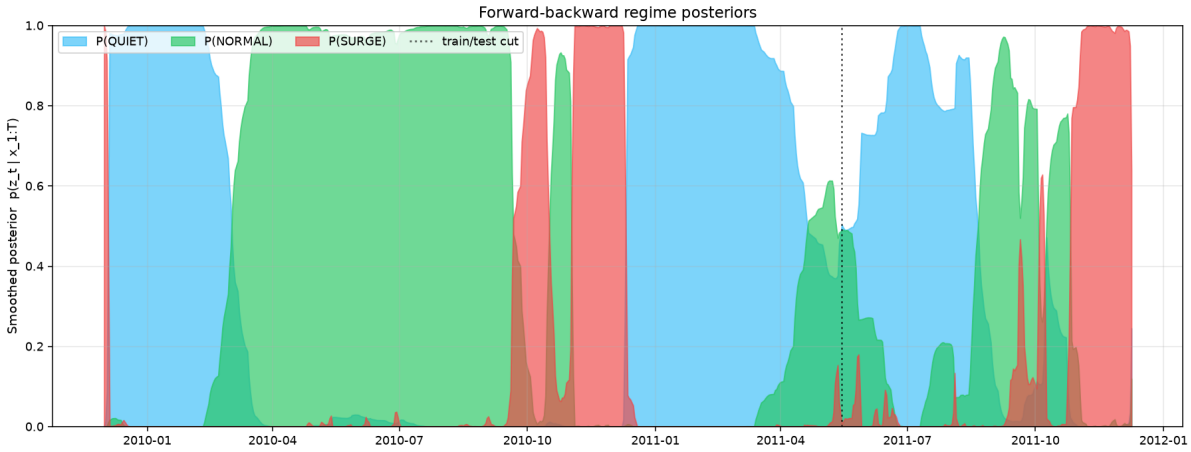
The LDS is the HMM with a continuous hidden state. Here the hidden state is the true underlying demand *level*, which drifts and is never observed directly because each day's sales are that level plus a large amount of transaction noise. A scalar local-level model (A = C = 1) was fitted by maximum likelihood over a grid.

Quantity	Value	Reading
Process noise q	0.00273	How fast the level truly moves
Observation noise r	0.33101	Day-to-day transaction noise
Signal-to-noise q/r	0.0083	Noise dominates by two orders of magnitude
Steady-state Kalman gain K	**0.087**	Also the alpha of the equivalent exponential smoother
Held-out one-step RMSE, Kalman	0.5917	log1p scale
Held-out one-step RMSE, naive random walk	0.7914	The benchmark to beat
Improvement	**+25.2%**	On data the model never saw
Smoothed variance never exceeds filtered	**True**	Required: smoothing conditions on more data

The low gain (0.087) is the model's considered judgement that most of the day-to-day movement in sales is noise rather than a genuine shift in demand, so the level estimate should barely react to any single day. Crucially this smoothing constant is *estimated by maximum likelihood*, not chosen because it looked reasonable. Only the filtered estimate and the one-step predictions are used as decision inputs; the RTS smoother uses future data and is for retrospective analysis.



Kalman filtered and RTS-smoothed demand level, and the Viterbi-decoded regimes. SURGE (red) lands on the Christmas peak in both years.



Forward-backward smoothed regime posteriors. Sharp blocks, not noise: the regimes persist for weeks.

4. Decision integration: what the two units change

A syllabus box ticked is not a contribution. Both units feed the procurement decision the project already makes.

4.1 Regime-aware safety stock (Unit 4)

regime	n days	share of days	mean demand	std demand	safety multiplier vs pooled
QUIET	254	42.1%	4,445	2,592	0.72x
NORMAL	261	43.2%	5,435	2,419	0.67x
SURGE	89	14.7%	10,229	5,124	1.42x

The actionable finding

Demand volatility differs **2.1x** between the calmest and the busiest regime. Sizing a buffer against the pooled standard deviation leaves it about 1.42x too small during SURGE — exactly when stockouts are most expensive — and about 0.67x oversized during QUIET, tying up working capital when it is least needed. The regime, not the average, is what a buffer should be sized against. The multipliers are computed from the **filtered** posterior, so they are available to a buyer on the morning of the decision.

4.2 Segment-aware policy (Unit 3)

The five segments are not cosmetic groupings. They separate 129 fast-moving daily sellers carrying 64% of revenue from 10 highly erratic SKUs (CV 9.3, average demand interval 42 days) and from 50 strongly Christmas-peaking items (Q4 ratio 14.1). Those three groups need different replenishment rules: a review period tuned for the first group will systematically over-order the second. The segment and its soft-assignment confidence are written to data/processed/sku_segments.parquet for the policy layer, and surfaced on dashboard page 7.

5. Verification

Three independent layers, because each catches what the others cannot.

Layer	What it checks	Result
Reference tests	From-scratch implementations against scikit-learn and SciPy	K-means inertia, PCA components, all 4 linkage methods, silhouette, Davies-Bouldin, Calinski-Harabasz and ARI all match
Brute-force tests	Dynamic programming against enumeration of all N^T state paths	Forward recursion, forward-backward posteriors and the Viterbi path all match exactly on a 3-state, 7-step HMM (2,187 paths)
Property tests	The mathematical guarantees the lectures state	EM monotonicity, EM bound tightness, rotational invariance of the FA likelihood, pi = pi A, smoothed variance never exceeding filtered
Browser verification	The dashboard actually renders, driven by Playwright	8/8 pages pass with zero exceptions

Test suite	Tests	Status
Units 1 and 2 (pre-existing)	81	passing
Unit 3 — unsupervised	79	passing
Unit 4 — sequential	79	passing
Total	**239**	**all passing**

The Playwright check matters more than it sounds. Streamlit traps exceptions per widget and renders them as a red box inside a running page, so a broken chart produces an app that starts perfectly and is wrong. The verifier launches the real server, clicks through all eight pages in Chromium, scrolls each to the bottom to force lazy widgets to render, and fails on any Streamlit exception, any error alert, or any missing expected heading. It also counts the tables, charts and figures that actually drew — a page with no exceptions but no content is still broken.

6. Honest limitations

- **Unit 5 is not covered.** The Google Drive link provided for it returns HTTP 401 without authentication, so its syllabus could not be read. Only the Unit 3 and Unit 4 decks were available locally.
- **The HMM is fitted on the aggregate series, not per SKU.** Business-wide regimes are a reasonable first model and are what the Christmas peak actually is, but an individual SKU can be in its own promotion while the business is QUIET.
- **Factor analysis converges slowly.** 2,378 EM iterations at tol 1e-8; a tighter tolerance needs roughly 8,200 for another 0.06 of log-likelihood. The tolerance was chosen by measuring that knee rather than by guessing.
- **The scree-elbow rule returns 1 component** on this spectrum. It is reported as-is rather than quietly dropped, with Kaiser used for the actual choice.
- **Regime-aware safety stock is quantified, not yet A/B tested end to end.** The multipliers are derived from held-out filtered posteriors, but a full re-run of the policy ladder under regime-conditioned buffers is the natural next step.
- **The supply side remains simulated.** Stock levels, lead times, MOQ and costs are not in the source dataset; this is unchanged from the existing project and is flagged throughout the dashboard.